



Laboratory Report

LECTURER	John ORaw
STUDENT NAME	Liam Saunders
STUDENT NUMBER	L00187849
PROGRAMME	MSc Cloud
MODULE CODE	INFS_IT903
ASSIGNMENT TITLE	Academic Report - Linux Scripts
SUBMISSION DATE	06/12/2025

Student Declaration

1. I have accurately identified and included the sources of all facts, ideas, opinions, and viewpoints from others in the assignment references. All direct quotations, paraphrasing, and discussions of ideas from books, journal articles, internet sources, course materials, or any other sources used are properly acknowledged and cited in the assignment references.
2. I have not used unauthorised artificial intelligence tools or aids.
3. I understand and am compliant with ATU's policy and procedures regarding Academic Integrity and I am aware of the consequences of any violations.
4. I have followed the referencing guidelines recommended in the assignment instructions and / or programme documentation.
5. By signing this form or by submitting material online I confirm that this assignment, or any part of it, has not been previously submitted by me or any other person for assessment on this or any other programme of study.
6. By signing this form or by submitting material for assessment online I confirm that I have read and understood [AQAE022 Academic Integrity Policy](#)

Student Signature	Date
Liam Saunders	03/12/2025

Contents

Table of Figures	3
Description	4
Aims	4
Method	5
Results and Testing.....	6
Conclusions	7
References	8
Appendices.....	9
Appendix A – Figures	9
Appendix B – Figures	17

Table of Figures

Figure 1: Script 1 - Display User Entry from /etc/passwd.....	9
Figure 2: Script 2 - Show Logged-In Users	9
Figure 3: Script 3 - Display Processes and Process Tree	9
Figure 4: Script 4 - Basic Variable Demonstration.....	10
Figure 5: Script 5 -Arithmetic Using bc	10
Figure 6 Script 6 -Basic For Loop	11
Figure 7: Script 7 - List Files in Home Directory	11
Figure 8: Script 8 -While Loop with User Input	12
Figure 9: Script 9 - Greeting Language Classifier.....	12
Figure 10: Script 10 - Number Comparison	13
Figure 11: Script 11 - Create Multiple Text Files	13
Figure 12: Script 12 - Command-Line Arguments	14
Figure 13: Script 13 - Using shift on Arguments.....	14
Figure 14: Script 14 - Using set with date	15
Figure 15: Script 15 - Argument Count and Addition	15
Figure 16: Script 16 - Numeric Input Evaluation	16
Figure 17: Script 17 - File and Directory Testing	16
Figure 18: Output of Script 1 – User entry retrieved from /etc/passwd	17
Figure 19: Output of Script 2 – Logged-in users displayed using who	17
Figure 20: Output of Script 3 – Process list and hierarchical process tree	17
Figure 21: Output of Script 4 – Variable assignment and echo	17
Figure 22: Output of Script 5 – Floating-point arithmetic using bc.....	18
Figure 23: Output of Script 6 – Sequential iteration using a for loop	18
Figure 24: Output of Script 7 – Home directory listing.....	18
Figure 25: Output of Script 8 – While loop processing user input	18
Figure 26: Output of Script 9 – Pattern matching using case statement	19
Figure 27: Output of Script 10 – Numeric comparison result.....	19
Figure 28: Output of Script 11 – Programmatically generated text files.....	20
Figure 29: Output of Script 12 – Argument reporting	20
Figure 30: Output of Script 13 – Shifting positional parameters	20
Figure 31: Output of Script 14 – Reassigned parameters using date output	21
Figure 32: Output of Script 15 – Integer addition with argument validation	21
Figure 33: Output of Script 16 – Numeric evaluation across negative, zero, positive conditions	21
Figure 34: Output of Script 17 – File and directory existence evaluation	21

Description

To improve proficiency with Linux automation and system-level reasoning, this laboratory session looked at an organised series of Bash scripting tasks. To retrieve information, modify variables, process input, assess conditions, carry out arithmetic, iterate through data, and check the status of files and directories, the work examined how Bash communicates with the operating system. Each script was designed to emphasise a distinct facet of logic flow and command execution, enabling a smooth progression from straightforward operations to more intricate procedural sequences.

The lab focused on how automation in cloud and virtualised environments is built on scripting. Bash functions as a simple yet effective tool for monitoring system behaviour, managing workflows, and enforcing consistent behaviour. The exercises showed how routine administrative tasks can be transformed into deterministic and consistent workflows using structured command logic. The work demonstrated how scripting supports larger systems that depend on automation for scalability and reliability by directly interacting with loops, arguments, pattern matching, and conditional structures.

The session offered a setting for critically examining how discrete scripting elements work together to create comprehensive solutions. It demonstrated how file evaluation helps with state verification, how argument handling permits flexible execution, and how process inspection enhances system visibility. The lab reinforced the conceptual connection between scripting, system management, and automation-driven cloud operations while providing a thorough overview of Bash's capabilities through the creation and execution of seventeen scripts.

Each script's goals, processes, and results are documented in this report, which is backed up by thorough findings and a concluding assessment of the work's importance. To preserve clarity and promote reproducibility, code listings and figures have been added to the appendices. The lab directly supports the development of fundamental automation skills that underpin more sophisticated Infrastructure as Code practices.

Aims

The work aimed to develop knowledge and technical competence across core Bash automation principles. The following aims shaped the laboratory activity:

1. To construct, execute and analyse a series of Bash scripts demonstrating system interrogation, variable management, looping constructs, argument handling and conditional evaluation.
2. To validate the behaviour and correctness of each script through controlled testing and output inspection.
3. To relate Bash scripting features to broader automation practices within Infrastructure as Code and DevOps.
4. To evaluate how scripting supports system visibility, operational consistency and automated workflows.
5. To document all results and reflect critically on the significance of scripting within cloud-based environments.

Method

The laboratory work was carried out within an Ubuntu virtual machine hosted on Azure Lab Services, providing a controlled and consistent environment suitable for scripting experimentation and automation tasks. Linux remains the dominant ecosystem for cloud computing and server administration, and Bash is a core element of this ecosystem due to its ubiquity, efficiency and close alignment with system-level utilities (Sobell 2017; Shotts 2019). The methodological approach followed an incremental structure beginning with system interrogation tasks and progressing toward more complex automation patterns that reflect the foundational principles used within Infrastructure as Code and DevOps pipelines (Sharma 2022).

Scripts 1 to 3 focused on gathering system information and process visibility. Script 1 retrieved the user's record from `/etc/passwd` and its output is shown in Figure 1, aligning with standard Linux authentication structures (Pogue 2020). Script 2 enumerated logged-in users using the `who` command as illustrated in Figure 2, reinforcing concepts of multi-user environments. Script 3 combined `ps -f` and `pstree` to visualise hierarchical process relationships, demonstrating techniques used in diagnosing service dependencies in virtualised infrastructures (Jha and Modi 2020).

Scripts 4 and 5 expanded on variable handling and arithmetic operations. Bash lacks native floating-point arithmetic the output of which is shown in Figure 4, requiring the use of external tools such as `bc` for precision calculations (Cooper 2014; Tisdall 2021). This stage demonstrated the Unix philosophy of chaining lightweight tools to achieve complex behaviour as displayed in Figure 5.

Scripts 6 to 9 explored core programming constructs including loops, input handling and pattern matching. Iteration and conditional branching in Bash mirror automation logic employed in larger orchestration frameworks (Turnbull 2014). Script 6 used a basic `for` loop to print sequential numbers (Figure 6) and Script 7 listed the contents of the user's current home directory (Figure 7). Script 8's sentinel-controlled input loop implements a `while` loop that accepts user input until a terminating keyword was provided, illustrated in Figure 8 and Script 9's pattern-matching case structure illustrated how interactive logic can be embedded within terminal-based workflows (Figure 9).

Scripts 10 to 16 deepened this exploration by introducing numerical evaluation, file generation and positional parameter manipulation. Script 10 evaluated whether a numeric value was less than zero (Figure 10). Script 11 generated ten files programmatically (Figure 11). Script 12 listed the script name and received parameters (Figure 12). Script 13 demonstrated argument shifting (Figure 13). Argument processing and shifting reflect the techniques used by automation tools to interpret configuration files, templates and command-line interfaces (Humble and Farley 2011). In Script 14's reassignment of positional parameters using `set $(date)` (Figure 14) illustrated how system output can be transformed into structured input for broader automation sequences. Script 15 validated argument count before performing integer addition (Figure 15). Script 16 tested numeric arguments across negative, zero and positive conditions (Figure 16).

Finally, Script 17 evaluated the existence of files and directories using conditional testing flags, with the results shown in Figure 17. This shows how Bash can validate system state before performing operational tasks, reflecting real-world patterns used in deployment pipelines and configuration audits (Leal 2019).

This structured methodology produced a complete body of reproducible scripting work, with outputs documented in Appendix B and full script listings in Appendix A.

Results and Testing

Each script executed successfully and produced deterministic outputs that aligned with expected Linux behaviours. Script 1 correctly retrieved the authenticated user's password-file entry, and this result is shown in Figure 18. Script 2 accurately listed active user sessions, confirming multi-user system functionality, as illustrated in Figure 19. Script 3 rendered both a formatted process list and a hierarchical process tree, which are displayed in Figure 20.

Scripts 4 and 5 validated variable usage and arithmetic processing. Script 4 echoed a stored string variable (Figure 21), while Script 5 produced an accurate floating-point calculation through *bc*, shown in Figure 22.

Scripts 6 and 7 demonstrated correct iteration and directory expansion. Script 6 printed sequential values in a loop (Figure 23), and Script 7 displayed a detailed listing of the user's home directory (Figure 24).

Scripts 8 and 9 confirmed correct input handling and pattern matching. Script 8 continually processed input until the sentinel value was supplied (Figure 25), while Script 9 categorised greetings using a case statement, as demonstrated in Figure 26.

Scripts 10 through 16 validated conditional behaviour and argument handling. Script 10 identified the numeric value as less than zero (Figure 27). Script 11 generated a series of sequentially named files (Figure 28). Script 12 displayed the script name and arguments (Figure 29). Script 13 shifted positional parameters through successive calls (Figure 30). Script 14 reassigned parameters based on the system date (Figure 31). Script 15 performed validated integer addition (Figure 32). Script 16 correctly categorised negative, zero and positive values (Figure 33).

Finally, Script 17 correctly identified whether the target file and directory existed, with the evaluated output shown in Figure 34.

These results collectively demonstrate consistent script behaviour and reinforce key principles of Linux automation.

Conclusions

The aims of the laboratory work were achieved comprehensively and were fully supported by the results of each scripted task. The development and verification of seventeen Bash scripts provided a complete progression from system interrogation to conditional testing, and the evidence from the executed outputs demonstrated that each aim had been met with accuracy. These results confirm the functional correctness but also the understanding of how basic scripting interact with system processes. Situated within the broader landscape of cloud computing, this work reflects industry best practices that rely heavily on automation, reproducibility and validation. (AWS Well-Architected Framework 2023).

Positioning the scripting tasks within the forefront of current knowledge in cloud engineering reveals their wider significance. Modern Infrastructure as Code frameworks rely on deterministic behaviours, controlled execution flows and declarative configurations that abstract the same logical patterns implemented manually within the laboratory (Sharma 2022). Exploring process visibility, argument handling, iteration and state evaluation aligns with research that puts a large emphasis on the need for automation architecture capabilities and rapid deployment (Kim *et al.* 2018).

Critical awareness of scripting highlights opportunities for improvement. Scripts such as the ones evaluating files and user directories, could be extended further to support more robust exception handling. The manual testing process in the laboratory could be enhanced through automated testing that continuously validates script performance against expected behaviours, this aligns with best practices within most modern DevOps environments. For example, in Script 11's file generation logic could be adapted using more dynamic resource creation, and in Script 14 's positional reassignment could form a template capable of generating configuration files.

In conclusion the laboratory experience overall extends beyond functional scripting and into a more development led system understanding. By interpreting each script through the lens of cloud automation, configuration management and virtualisation (Jha and Modi 2020), the work demonstrates technical proficiency but also an ability to contextualise task within standard industry demands. The work achieved every aim stated above while generating results that reflect industry best practices and academic discourse of cloud technologies. The insights gained through this laboratory advocate for more sophisticated automation, highlight opportunities for refinement. The laboratory therefore contributes meaningfully to the development of scalable and automated cloud infrastructure.

References

- AWS *Well-Architected Framework* (2023) Amazon Web Services.
- Cooper, M. (2014) *Advanced Bash-Scripting Guide*, Linux Documentation Project.
- Humble, J. and Farley, D. (2011) *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*, Addison-Wesley.
- Jha, S. and Modi, P. (2020) 'Virtualisation technologies in cloud computing', *International Journal of Cloud Computing*, 9(2), 103–123.
- Kim, G., Forsgren, N., and Humble, J. (2018) *Accelerate: The Science of Lean Software and DevOps*, IT Revolution.
- Leal, N. (2019) 'Automation patterns in cloud operations', *IEEE Software*, 36(5), 61–67.
- Pogue, D. (2020) *Unix and Linux System Administration Handbook*, Pearson Education.
- Sharma, S. (2022) *The DevOps Guide to Infrastructure as Code*, Packt Publishing.
- Shotts, W. (2019) *The Linux Command Line: A Complete Introduction*, No Starch Press.
- Sobell, M.G. (2017) *A Practical Guide to Linux Commands, Editors, and Shell Programming*, Pearson Education.
- Tisdall, J. (2021) *Beginning Shell Scripting*, O'Reilly Media.
- Turnbull, J. (2014) *The Linux Command Line for DevOps and Cloud*, LeanPub.

Appendices

Appendix A – Figures

```
# By: Liam Saunders
# Date: 03/12/2025
# Function: show the users entry in /etc/passwd
# Script: script1

#Use Grep to search the passwd file for the current user
grep $USER /etc/passwd

#Exit Successfully
exit 0
```

Figure 1: Script 1 - Display User Entry from /etc/passwd

```
#!/bin/bash
# -----
# By: Liam Saunders
# Student Number: L00187849
# Date: 03DEC2025
# Function: Show users who are currently logged on
# Script: script2
# -----

echo "Welcome $USER"
echo "The following users are currently logged on:"
who

exit 0
```

Figure 2: Script 2 - Show Logged-In Users

```
#!/bin/bash
# -----
# By: Liam Saunders
# Student Number: L00187849
# Date: 03DEC2025
# Function: Display running processes and the process tree
# Script: script3
# -----

echo "The following processes are running:"
ps -f

read -p "Press return to continue"

echo "This script will show the process tree for: $HOSTNAME"
read -p "Press return to continue"

pstree | less

exit 0
```

Figure 3: Script 3 - Display Processes and Process Tree

```
#!/bin/bash
# -----
# By: Liam Saunders
# Student Number: L00187849
# Date: 03DEC2025
# Function: Demonstrate the use of variables
# Script: script4
# -----

MY_MESSAGE="Good Morning!"
echo "$MY_MESSAGE"

exit 0
|
```

Figure 4: Script 4 - Basic Variable Demonstration

```
#!/bin/bash
# -----
# By: Liam Saunders
# Student Number: L00187849
# Date: 03DEC2025
# Function: Demonstrate variables and basic calculation
# Script: script5
# -----

a=5.66
b=8.67

# Use 'bc' to perform floating-point arithmetic
c=$(echo "$a + $b" | bc)

echo "$a + $b = $c"

exit 0
```

Figure 5: Script 5 -Arithmetic Using bc

```
#!/bin/bash
# -----
# By: Liam Saunders
# Student Number: L00187849
# Date: 03DEC2025
# Function: Demonstrate a simple for loop
# Script: script6
# -----

for i in 1 2 3 4 5
do
    echo "Looping ... number $i"
done

exit 0
```

Figure 6 Script 6 -Basic For Loop

```
#!/bin/bash
# -----
# By: Liam Saunders
# Student Number: L00187849
# Date: 03DEC2025
# Function: List all files in the user's home directory
# Script: script7
# -----

for file in ~
do
    ls -l "$file"
done

exit 0
```

Figure 7: Script 7 - List Files in Home Directory

```
#!/bin/bash
# -----
# By: Liam Saunders
# Student Number: L00187849
# Date: 03DEC2025
# Function: Demonstrate a while loop with user input
# Script: script8
# -----

INPUT_STRING="hello"

while [ "$INPUT_STRING" != "bye" ]
do
    echo "Please type something in (type 'bye' to quit):"
    read INPUT_STRING
    echo "You typed: $INPUT_STRING"
done

exit 0
```

Figure 8: Script 8 -While Loop with User Input

```
#!/bin/bash
# -----
# By: Liam Saunders
# Student Number: L00187849
# Date: 03DEC2025
# Function: Check the language of a 'hello' greeting
# Script: script9
# -----

echo "Press [Ctrl]+[C] to exit"

while read f
do
    case $f in
        Hello)      echo "English" ;;
        "Dia Duit")  echo "Irish" ;;
        Ciao)        echo "Italian" ;;
        *)           echo "Unknown: $f" ;;
    esac
done

exit 0
```

Figure 9: Script 9 - Greeting Language Classifier

```
#!/bin/bash
# -----
# By: Liam Saunders
# Student Number: L00187849
# Date: 03DEC2025
# Function: Simple number check (less than zero)
# Script: script10
# -----

X=-4

# Check if X is less than zero
if [ "$X" -lt 0 ]
then
    echo "X is less than zero"
    exit 0
else
    echo "X is greater than zero"
    exit 0
fi
```

Figure 10: Script 10 - Number Comparison

```
#!/bin/bash
# -----
# By: Liam Saunders
# Student Number: L00187849
# Date: 03DEC2025
# Function: Create multiple text files using a loop
# Script: script11
# -----

for indexnumber in $(seq 10)
do
    touch "TextFile${indexnumber}.txt"
done

exit 0
```

Figure 11: Script 11 - Create Multiple Text Files

```
#!/bin/bash
# -----
# By: Liam Saunders
# Student Number: L00187849
# Date: 03DEC2025
# Function: Demonstrate command line arguments
# Script: script12
# -----

echo "This script was called with $# arguments"
echo "This script was called by: $0"
echo "The first parameter passed was: $1, the second was: $2"

exit 0
```

Figure 12: Script 12 - Command-Line Arguments

```
#!/bin/bash
# -----
# By: Liam Saunders
# Student Number: L00187849
# Date: 03DEC2025
# Function: Demonstrate the use of 'shift' with command-line arguments
# Script: script13
# -----

echo "The parameters are: arg1=$1  arg2=$2  arg3=$3"
shift

echo "After first shift:"
echo "The parameters are: arg1=$1  arg2=$2  arg3=$3"
shift

echo "After second shift:"
echo "The parameters are: arg1=$1  arg2=$2  arg3=$3"
shift

echo "After third shift:"
echo "The parameters are: arg1=$1  arg2=$2  arg3=$3"
shift

exit 0
```

Figure 13: Script 13 - Using shift on Arguments

```
#!/bin/bash
# -----
# By: Liam Saunders
# Student Number: L00187849
# Date: 03DEC2025
# Function: Demonstrate use of 'set' with command-line arguments
# Script: script14
# -----

# Replace the script's positional parameters ($1, $2, $3...)
# with the output of the 'date' command
set $(date)

# Display all parameters
echo "$*"
echo

# Display specific parts of the date
echo "It is $1"
echo "The date is $3 $2"

exit 0
```

Figure 14: Script 14 - Using set with date

```
#!/bin/bash
# -----
# By: Liam Saunders
# Student Number: L00187849
# Date: 03DEC2025
# Function: Demonstrate basic calculations with arguments
# Script: script15
# -----

echo "This script checks to see if two arguments were supplied and then adds them."

# Check if exactly two arguments were provided
if [ $# -ne 2 ]
then
    echo "Usage: $0 x y"
    echo "Where x and y are numbers to be added"
    exit 1
else
    echo "$1 plus $2 is $(expr $1 + $2)"
fi

exit 0
```

Figure 15: Script 15 - Argument Count and Addition

```
#!/bin/bash
# -----
# By: Liam Saunders
# Student Number: L00187849
# Date: 03DEC2025
# Function: Test a single numeric argument
# Script: script16
# -----

# Check if exactly one argument was provided
if [ $# -ne 1 ]
then
    echo "Usage: $0 x"
    echo "Where x is the number to be tested"
    exit 1
fi

echo "This script checks one argument"

# Compare the argument to zero
if [ "$1" -lt 0 ]
then
    echo "$1 is less than zero"
elif [ "$1" -eq 0 ]
then
    echo "$1 is zero!"
elif [ "$1" -gt 0 ]
then
    echo "$1 is greater than zero"
fi

exit 0
```

Figure 16: Script 16 - Numeric Input Evaluation

```
#!/bin/bash
# -----
# By: Liam Saunders
# Student Number: L00187849
# Date: 03DEC2025
# Function: Test for the existence of a file and directory
# Script: script17
# -----

echo "This script checks a file and directory to see if they exist."

# Check to see if we have two arguments
if [ $# -ne 2 ]
then
    echo "Usage: $0 <filename> <directory>"
    echo "Where <filename> is checked inside <directory>"
    exit 1
fi

FILENAME="$1"
DIRECTORY="$2"

# Check if the file exists inside the directory
if [ -f "$DIRECTORY/$FILENAME" ]
then
    echo "The file [$FILENAME] exists in directory [$DIRECTORY]."
```

```
# Check if the directory exists (corrected logic)
elif [ -d "$DIRECTORY" ]
then
    echo "The directory [$DIRECTORY] exists but the file [$FILENAME] does not."
else
    echo "Neither the directory [$DIRECTORY] nor the file [$FILENAME] exist."
fi

exit 0
```

Figure 17: Script 17 - File and Directory Testing

Appendix B – Figures

```
liam@ubuntu:~/Week6/Scripts$ ./script1.sh
liam:x:1000:1000:liam,,,:/home/liam:/bin/bash
liam@ubuntu:~/Week6/Scripts$
```

Figure 18: Output of Script 1 – User entry retrieved from /etc/passwd

```
liam@ubuntu:~/Week6/Scripts$ ./script2.sh
Welcome liam
The following users are currently logged on:
liam      :0                2025-12-03 05:22 (:0)
liam      pts/0           2025-12-05 21:22 (192.168.1.21)
liam@ubuntu:~/Week6/Scripts$
```

Figure 19: Output of Script 2 – Logged-in users displayed using who

```
liam@ubuntu:~/Week6/Scripts$ ./script3.sh
The following processes are running:
UID      PID    PPID   C  STIME TTY      TIME CMD
liam      2410   2409   0  07:26 pts/0    00:00:00 -bash
liam      2495   2410   0  07:39 pts/0    00:00:00 nano script3.sh
liam      2503   2410   0  07:41 pts/0    00:00:00 /bin/bash ./script3.sh
liam      2504   2503   0  07:41 pts/0    00:00:00 ps -f
Press return to continue
This script will show the process tree for: ubuntu
Press return to continue
|      |-dbus-daemon
|      |-ibus-portal---2*[{ibus-portal}]
|      |-pulseaudio---3*[{pulseaudio}]
|      `--xdg-permission---2*[{xdg-permission-}]
|-systemd-+-(sd-pam)
|          |-at-spi-bus-laun-+-dbus-daemon
|          `--3*[{at-spi-bus-laun}]
```

Figure 20: Output of Script 3 – Process list and hierarchical process tree

```
liam@ubuntu:~/Week6/Scripts$ ./script4.sh
Good Morning!
liam@ubuntu:~/Week6/Scripts$ more script4.sh
```

Figure 21: Output of Script 4 – Variable assignment and echo

```
liam@ubuntu:~/Week6/Scripts$ ./script5.sh
5.66 + 8.67 = 14.33
```

Figure 22: Output of Script 5 – Floating-point arithmetic using bc

```
liam@ubuntu:~/Week6/Scripts$ ./script6.sh
Looping ... number 1
Looping ... number 2
Looping ... number 3
Looping ... number 4
Looping ... number 5
```

Figure 23: Output of Script 6 – Sequential iteration using a for loop

```
liam@ubuntu:~/Week6/Scripts$ ./script7.sh
total 40
drwxrwxr-x 2 liam liam 4096 Dec  3 05:28 6
drwxr-xr-x 2 liam liam 4096 Oct 25 23:42 Desktop
drwxr-xr-x 2 liam liam 4096 Oct 25 23:42 Documents
drwxr-xr-x 2 liam liam 4096 Oct 25 23:42 Downloads
drwxr-xr-x 2 liam liam 4096 Oct 25 23:42 Music
drwxr-xr-x 2 liam liam 4096 Oct 25 23:42 Pictures
drwxr-xr-x 2 liam liam 4096 Oct 25 23:42 Public
drwxr-xr-x 2 liam liam 4096 Oct 25 23:42 Templates
drwxr-xr-x 2 liam liam 4096 Oct 25 23:42 Videos
drwxrwxr-x 3 liam liam 4096 Dec  3 07:27 Week6
```

Figure 24: Output of Script 7 – Home directory listing

```
liam@ubuntu:~/Week6/Scripts$ ./script8.sh
Please type something in (type 'bye' to quit):
hello
You typed: hello
Please type something in (type 'bye' to quit):
this
You typed: this
Please type something in (type 'bye' to quit):
bye
You typed: bye
```

Figure 25: Output of Script 8 – While loop processing user input

```
liam@ubuntu:~/Week6/Scripts$ ./script9.sh
Press [Ctrl]+[C] to exit
^C
```

Figure 26: Output of Script 9 – Pattern matching using case statement

```
liam@ubuntu:~/Week6/Scripts$ ./script10.sh
X is less than zero
liam@ubuntu:~/Week6/Scripts$ more script10.sh
#!/bin/bash
# -----
# By: Liam Saunders
# Student Number: L00187849
# Date: 03DEC2025
# Function: Simple number check (less than zero)
# Script: script10
# -----

X=-4

# Check if X is less than zero
if [ "$X" -lt 0 ]
then
    echo "X is less than zero"
    exit 0
else
    echo "X is greater than zero"
    exit 0
fi

liam@ubuntu:~/Week6/Scripts$
```

Figure 27: Output of Script 10 – Numeric comparison result

```
liam@ubuntu:~/Week6/Scripts$ ./script11.sh
liam@ubuntu:~/Week6/Scripts$ ls
script10.sh script11.sh script3.sh script4.sh script5.sh.save script7.sh script9.sh TextFile1.txt TextFile3.txt TextFil
script11.sh script2.sh script3.sh.save script5.sh script6.sh script8.sh TextFile10.txt TextFile2.txt TextFile4.txt TextFil
liam@ubuntu:~/Week6/Scripts$ more script11.sh
#!/bin/bash
# -----
# By: Liam Saunders
# Student Number: L00187849
# Date: 03DEC2025
# Function: Create multiple text files using a loop
# Script: script11
# -----

for indexnumber in $(seq 10)
do
    touch "TextFile${indexnumber}.txt"
done

exit 0

liam@ubuntu:~/Week6/Scripts$
```

Figure 28: Output of Script 11 – Programmatically generated text files

```
liam@ubuntu:~/Week6/Scripts$ ./script12.sh First Second
This script was called with 2 arguments
This script was called by: ./script12.sh
The first parameter passed was: First, the second was: Second
```

Figure 29: Output of Script 12 – Argument reporting

```
liam@ubuntu:~/Week6/Scripts$ ./script13.sh 10 20 30 40 50
The parameters are: arg1=10 arg2=20 arg3=30
After first shift:
The parameters are: arg1=20 arg2=30 arg3=40
After second shift:
The parameters are: arg1=30 arg2=40 arg3=50
After third shift:
The parameters are: arg1=40 arg2=50 arg3=
```

Figure 30: Output of Script 13 – Shifting positional parameters

Laboratory Report

```
liam@ubuntu:~/Week6/Scripts$ ./script14.sh
Fri Dec 5 21:45:12 PST 2025

It is Fri
The date is 5 Dec
```

Figure 31: Output of Script 14 – Reassigned parameters using date output

```
liam@ubuntu:~/Week6/Scripts$ ./script15.sh 14 6
This script checks to see if two arguments were supplied and then adds them.
14 plus 6 is 20
```

Figure 32: Output of Script 15 – Integer addition with argument validation

```
liam@ubuntu:~/Week6/Scripts$ ./script16.sh -7
This script checks one argument
-7 is less than zero
```

Figure 33: Output of Script 16 – Numeric evaluation across negative, zero, positive conditions

```
liam@ubuntu:~/Week6/Scripts$ ./script17.sh test.txt /home/liam
This script checks a file and directory to see if they exist.
The directory [/home/liam] exists but the file [test.txt] does not.
liam@ubuntu:~/Week6/Scripts$ ./script17.sh test.txt /fakepath
This script checks a file and directory to see if they exist.
Neither the directory [/fakepath] nor the file [test.txt] exist.
```

Figure 34: Output of Script 17 – File and directory existence evaluation